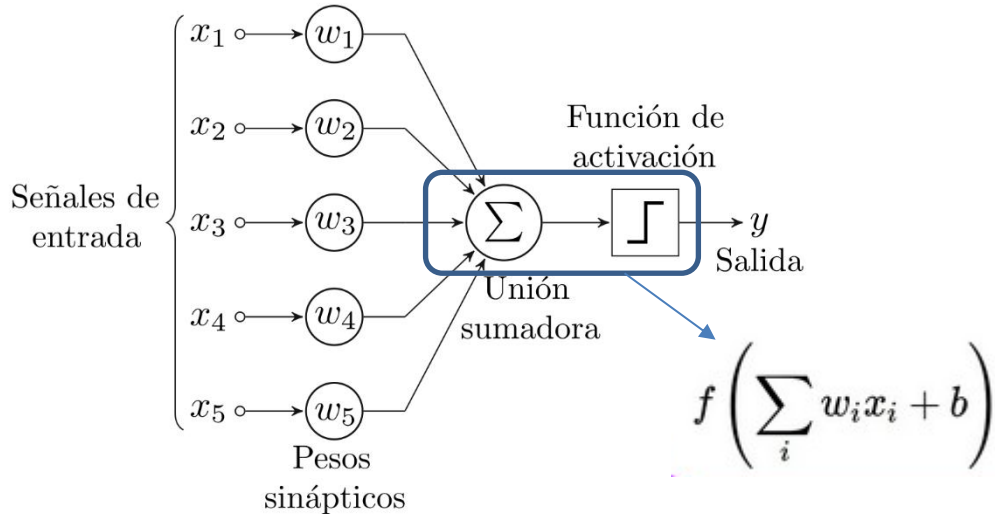


Presentación

En este tema se explicará cómo funciona una red neuronal de tipo MLP y se realizarán diferentes ejercicios prácticos. Antes de abordar el problema explicaremos qué es un perceptrón.

Perceptrón (I)

- Un perceptrón es la neurona artificial o unidad básica de inferencia en forma de discriminador lineal, a partir de lo cual se desarrolla un algoritmo capaz de generar un criterio para seleccionar un sub-grupo a partir de un grupo de componentes más grande.



- Es necesario entrenar el perceptrón para que se comporte como esperamos. El algoritmo de aprendizaje es el mismo para todas las neuronas, todo lo que sigue se aplica a una sola neurona en el aislamiento.
- El parámetro bias ayuda a controlar el valor que permitirá activar la neurona con la función de activación.
- Normalmente el parámetro bias se implementa como otra neurona con su propio peso.

Perceptrón (II)

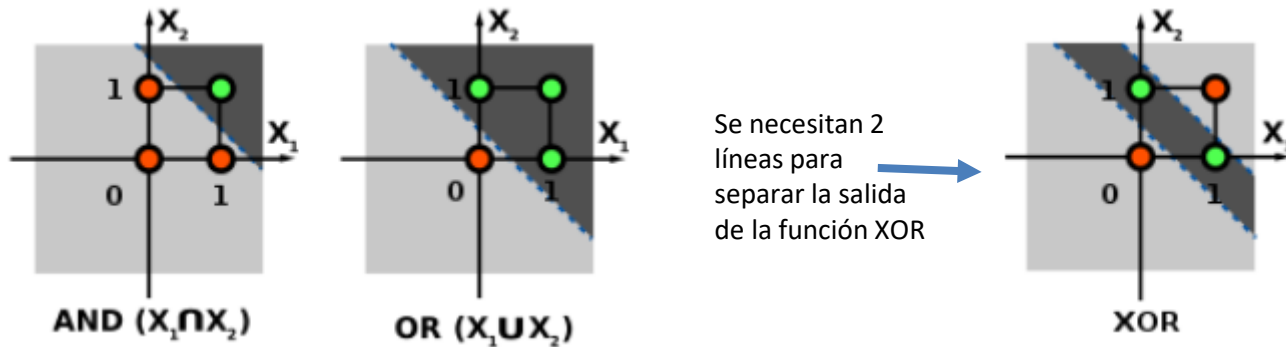
- Para el aprendizaje se definen una serie de variables:
 - $x(j)$ denota el elemento en la posición j en el vector de la entrada
 - $w(j)$ el elemento en la posición j en el vector de peso
 - y denota la salida de la neurona
 - δ denota la salida esperada
 - α es una constante tal que $0 < \alpha < 1$
- α representa la tasa de aprendizaje. Utilizando tasa de aprendizaje, utilizaremos la siguiente regla de actualización de los pesos:

$$w(j)' = w(j) + \alpha(\delta - y)x(j)$$

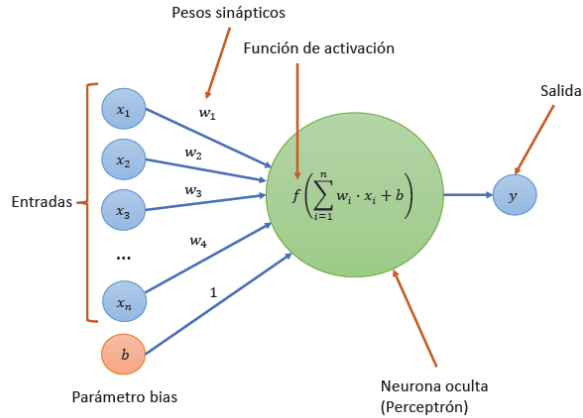
Esto es el error de dicha neurona

Perceptrón (III)

- Muchos problemas no se pueden resolver con un único perceptrón simple, debido a que las funciones que implementan no pueden separar sus resultados linealmente con una única línea.



Perceptrón (V)



Estado actual entrenamiento

$$x_1 = 0.5$$

$$w_1 = 0.8$$

$$b = 0.1$$

$$\text{resto } x_i = 0$$

$$\text{supongamos salida esperada} = 1.0$$

1. Paso hacia adelante (feed-forward)

1. **Cálculo de la salida del perceptrón:**

$$z = x \cdot w + b = 0.5 \cdot 0.8 + 0.1 = 0.4 + 0.1 = 0.5$$

2. **Aplicar la función de activación (sigmoide):**

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-0.5}} \approx 0.622$$

3. **Cálculo del error cuadrático:**

$$E = \frac{1}{2}(y - \hat{y})^2 = \frac{1}{2}(1 - 0.622)^2 = \frac{1}{2}(0.378)^2 \approx \frac{1}{2} \cdot 0.143 = 0.0715$$

Perceptrón (VI)

2. Paso hacia atrás (back-propagation)

4. **Retropropagación del error:** - **Derivada del error con respecto a la salida:**

$$\frac{\partial E}{\partial \hat{y}} = \hat{y} - y = 0.622 - 1 = -0.378$$

- **Derivada de la salida con respecto a z :**

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y}) = 0.622 \cdot (1 - 0.622) = 0.622 \cdot 0.378 \approx 0.235$$

- **Derivada de z con respecto al peso w :**

$$\frac{\partial z}{\partial w} = x = 0.5$$

- **Derivada de z con respecto al bias b :**

$$\frac{\partial z}{\partial b} = 1$$

- **Gradiente del error con respecto al peso w :**

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w} = -0.378 \cdot 0.235 \cdot 0.5 \approx -0.044$$

- **Gradiente del error con respecto al bias b :**

$$\frac{\partial E}{\partial b} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial b} = -0.378 \cdot 0.235 \cdot 1 \approx -0.089$$

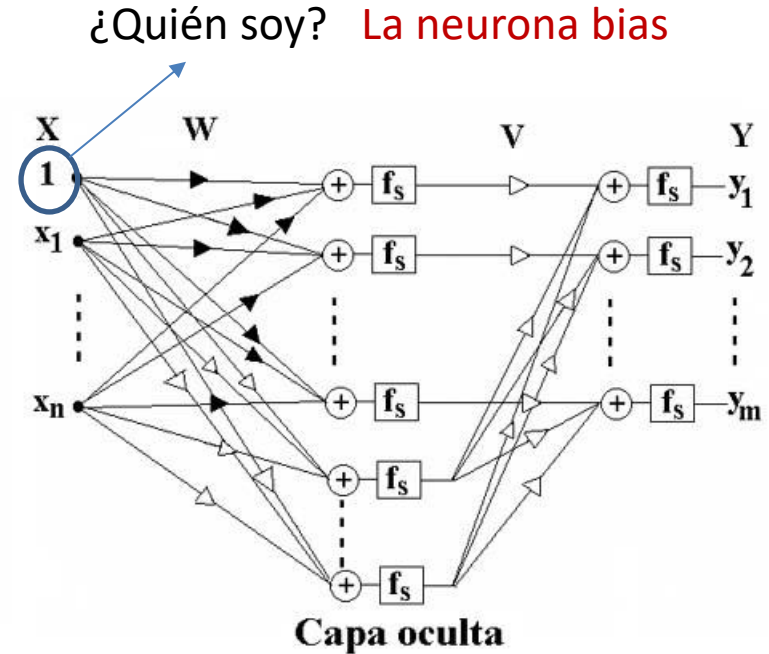
5. **Actualización de los pesos y el bias:** Si utilizamos una tasa de aprendizaje $\eta = 0.1$:

$$w_{\text{nuevo}} = w - \eta \cdot \frac{\partial E}{\partial w} = 0.8 - 0.1 \cdot (-0.044) = 0.8 + 0.0044 = 0.8044$$

$$b_{\text{nuevo}} = b - \eta \cdot \frac{\partial E}{\partial b} = 0.1 - 0.1 \cdot (-0.089) = 0.1 + 0.0089 = 0.1089$$

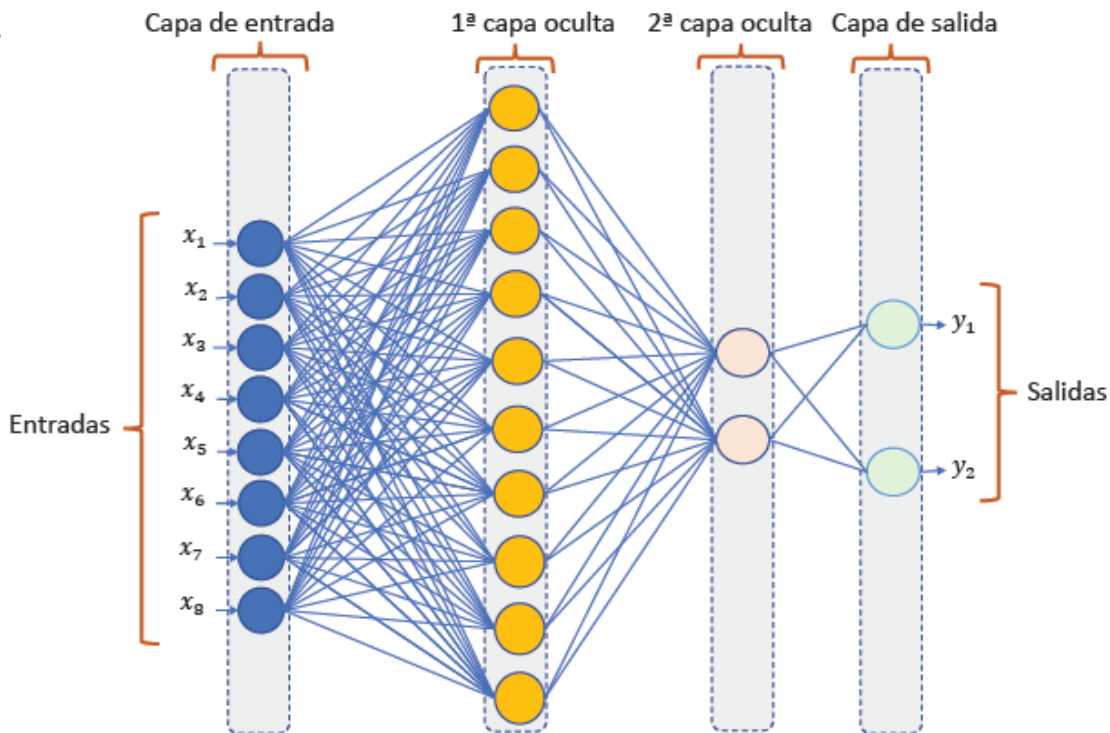
Perceptrón multicapa (I)

- Los problemas pueden tener varias neuronas e incluso varias capas de neuronas, convirtiéndose en perceptrones multicapa.
 - El aprendizaje en estos problemas se realiza mediante el algoritmo de retro-propagación, que sigue un método de descenso de gradiente:
 1. Se inicializan los pesos con unos valores iniciales (puede ser siguiendo una distribución normal).
 2. A partir de una muestra del conjunto de entrada, se calcula la salida realizando el proceso hacia adelante.
 3. Se calcula el error cuadrático medio de la salida.
 4. Si se alcanza un número de iteraciones o un error mínimo se para el proceso.
 5. Se va calculando la parte proporcional de error de cada neurona de salida a partir del error cuadrático medio del punto 3. A partir del error de cada neurona de salida se calculan los errores de las neuronas de la capa anterior.
 6. Se reajustan los pesos de cada neurona.
 7. Vuelta al paso 2.



Perceptrón multicapa (II)

- Multi-layer perceptron



Perceptrón Multicapa (III)

- Algoritmo de retropropagación (caso sigmoidal)

Procedimiento BACKPROPAGATION;

Inicializar los pesos aleatoriamente;

repetir

Poner entrada de entrenamiento;

FEED_FORWARD;

COMPUTO_GRADIENTE;

ACTUALIZA_PESOS;

hasta encontrar condición de final;

fin {BACKPROPAGATION}

Procedimiento FEED_FORWARD;

for capa=1 to L do

for nodo=1 to N_{capa} do $y_{capa,nodo} = f(\sum_{i=1}^{N_{capa-1}} w_{capa,nodo,i} \times y_{capa-1,i});$

end

end

end; {FEED_FORWARD}

Procedimiento COMPUTO_GRADIENTE;

for capa=L to 1 do

for nodo=1 to N_{capa} do

if (capa=L) then $e_{L,nodo} = y_{L,nodo} - d_{nodo};$

else

$$e_{capa,nodo} = \sum_{m=1}^{N_{capa+1}} e_{capa+1,m} y_{capa+1,m} (1 - y_{capa+1,m}) \times w_{capa+1,m,nodo};$$

end;

for todos los pesos en todas las capas do

$$g_{capa,j,i} = e_{capa,j} y_{capa,j} (1 - y_{capa,j}) y_{capa-1,i};$$

end;

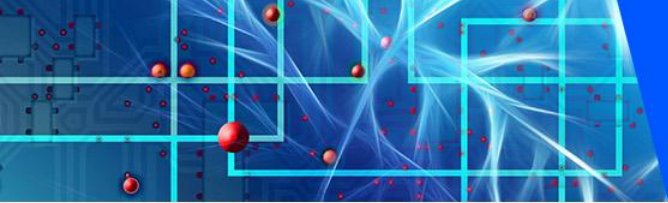
end; {COMPUTO_GRADIENTE}

Procedimiento ACTUALIZA_PESOS;

for todos los pesos $w_{l,j,i}$ do $w_{l,j,i}(k+1) = w_{l,j,i}(k) - \eta \cdot g_{l,j,i};$

end;

end; {ACTUALIZA_PESOS}



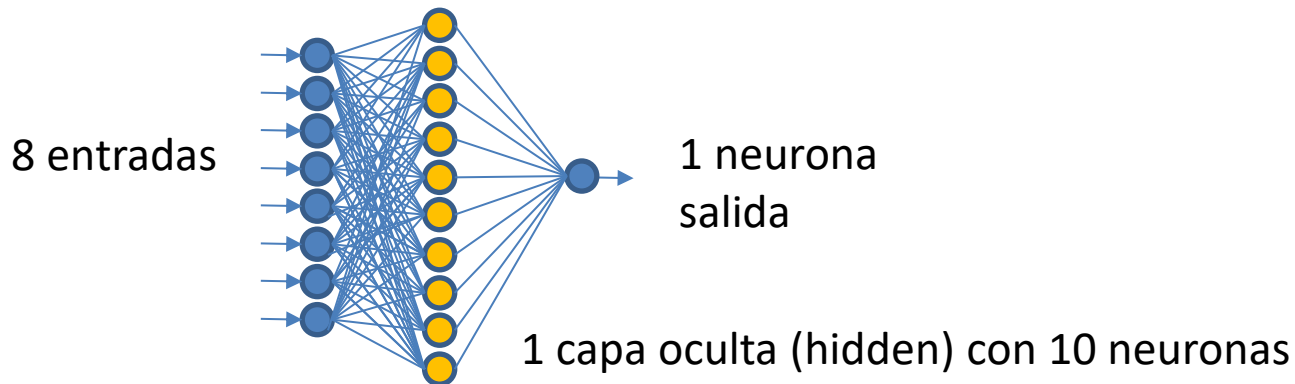
Deep Learning – Ciclo de Vida (I)

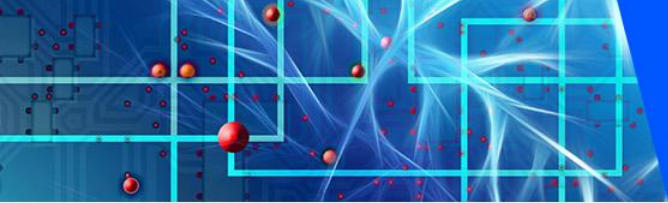
- Los cinco pasos del ciclo de vida del Deep Learning son los siguientes:
 - Definir el modelo.
 - Compilar el modelo.
 - Ajustar el modelo.
 - Evaluar el modelo.
 - Hacer predicciones.

Deep Learning – Ciclo de Vida (II)

- Definir el modelo. Ejemplo de modelo secuencial:

```
1 # example of a model defined with the sequential api
2 from tensorflow.keras import Sequential
3 from tensorflow.keras.layers import Dense
4 # define the model
5 model = Sequential()
6 model.add(Dense(10, input_shape=(8,)))
7 model.add(Dense(1))
```





Deep Learning – Ciclo de Vida (III)

- Definir el modelo. Ejemplo de modelo secuencial:

```
1 # example of a model defined with the sequential api
2 from tensorflow.keras import Sequential
3 from tensorflow.keras.layers import Dense
4 # define the model
5 model = Sequential()
6 model.add(Dense(100, input_shape=(8,)))
7 model.add(Dense(80))
8 model.add(Dense(30))
9 model.add(Dense(10))
10 model.add(Dense(5))
11 model.add(Dense(1))
```

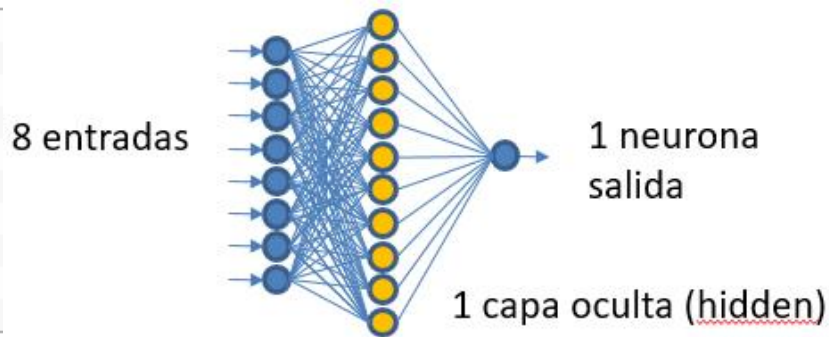
<https://machinelearningmastery.com/tensorflow-tutorial-deep-learning-with-tf-keras/>

Deep Learning – Ciclo de Vida (IV)

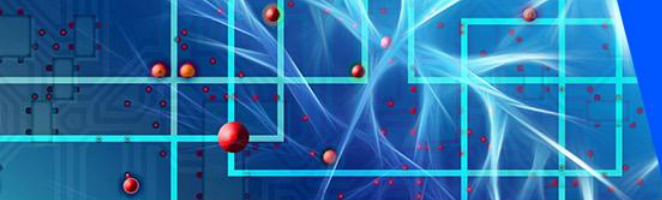
- Definir el modelo. Ejemplo de modelo funcional:

La API funcional permite más posibilidades que la secuencial ya que no obliga a la secuencialidad de una capa respecto a otra.

```
1 # example of a model defined with the functional api
2 from tensorflow.keras import Model
3 from tensorflow.keras import Input
4 from tensorflow.keras.layers import Dense
5 # define the layers
6 x_in = Input(shape=(8,))
7 x = Dense(10)(x_in)
8 x_out = Dense(1)(x)
9 # define the model
10 model = Model(inputs=x_in, outputs=x_out)
```



<https://machinelearningmastery.com/tensorflow-tutorial-deep-learning-with-tf-keras/>



Deep Learning – Ciclo de Vida (V)

- Compilar el modelo:

```
# compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

- Entrenar el modelo:

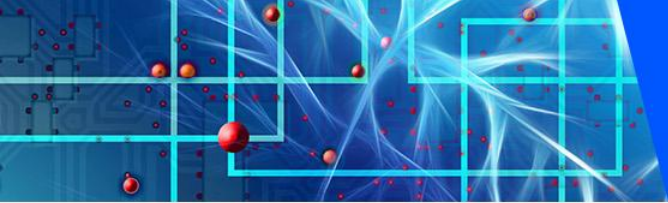
```
# fit the model
model.fit(X_train, y_train, epochs=150, batch_size=32, verbose=0)
```

- Evaluar el modelo:

```
# evaluate the model
loss, acc = model.evaluate(X_test, y_test, verbose=0)
```

- Hacer una predicción:

```
# make a prediction
row = [1,0,0.99539,-0.05889,0.85243,0.02306,0.83398,-0.37708,1,0.03760,0.85243,-0.
yhat = model.predict([row])
print('Predicted: %.3f' % yhat)
```



Deep Learning – Ciclo de Vida (VI)

- Algunas cuestiones (I):

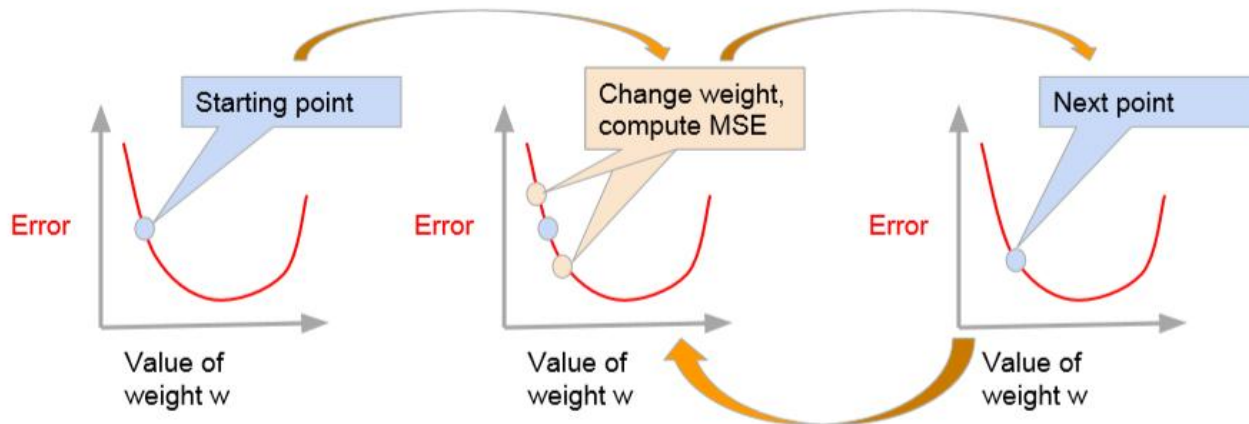
1. Durante el entrenamiento es necesario tener un juego de datos de entrenamiento, un juego de datos de validación y un juego de datos de test.
 - a. El conjunto de entrenamiento son los datos que entrenan el modelo. Normalmente actualiza los pesos o valores de los kernels de una CNN.
 - b. El conjunto de validación selecciona el mejor de los modelos entrenados. Permite ajustar los hiperparámetros de la red, como pueden ser el número de neuronas de capas intermedias o las distintas capas.
 - c. El conjunto de test ofrece el error real cometido con un modelo seleccionado. Se utiliza al finalizar el proceso para evaluar el modelo.
2. El entrenamiento de una red neuronal se realiza mediante algoritmos de descenso de gradiente estocástico que utilizan un conjunto de datos de entrenamiento para actualizar un modelo.
 - a. El tamaño del lote, o batch, es un hiperparámetro que indica el número de muestras de entrenamiento que hay que atravesar antes de que se actualicen los parámetros internos del modelo.
 - b. El número de épocas, o epochs, es un hiperparámetro de descenso de gradiente que controla el número de pases completos a través del conjunto de datos de entrenamiento.

Supongamos que tenemos un conjunto de datos de entrenamiento con 200 muestras (filas de datos) y elegimos número de batch = 5 y número de epochs = 1000. Esto significa que el conjunto de datos se dividirá en 40 lotes, cada uno con cinco muestras. Los pesos del modelo se actualizarán después de cada lote de cinco muestras. Esto también significa que una epoch implicará 40 lotes, es decir 40 actualizaciones del modelo. Con 1.000 epochs, el modelo pasará por todo el conjunto de datos 1.000 veces. En total 40.000 batch durante todo el proceso de entrenamiento.

Deep Learning – Ciclo de Vida (VII)

- Algunas cuestiones (II):

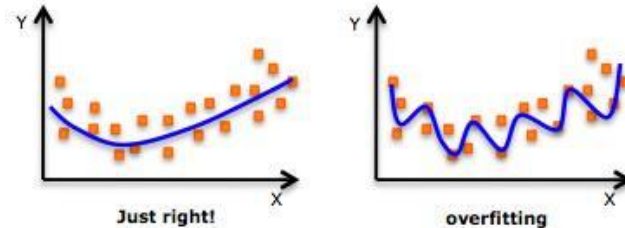
Recompute error after each batch of examples (not full dataset)



Deep Learning – Ciclo de Vida (VIII)

- Algunas cuestiones (III):

3. Overfitting: Ocurre cuando nuestro modelo se ajusta mucho a los datos de entrenamiento pero no funciona bien para otros datos diferentes. Uno de los motivos puede ser utilizar el juego de datos de entrenamiento exclusivamente.
 - Los síntomas son:
 - Bajo Bias (Predicciones muy ajustadas sobre el conjunto de entrenamiento)
 - Alta varianza (Pobre capacidad de predicción sobre el conjunto de validación)
 - Posible solución: Algunas redes se entrenan minimizando una función de error apropiada definida con respecto a un conjunto de datos de entrenamiento. A continuación, se compara el rendimiento de las redes evaluando la función de error mediante un conjunto de validación independiente, y se selecciona la red que tenga el menor error con respecto al conjunto de validación. Este enfoque se denomina método de retención “hold out method”. Dado que este procedimiento puede conducir por sí mismo a un overfitting respecto al conjunto de validación, el rendimiento de la red seleccionada debe confirmarse midiendo su rendimiento en un tercer conjunto de datos independiente denominado conjunto de test.



“Someone who learns the language using only Shakespeare, as an example, will learn a very specialized form of English and may not be able to understand other English text.”

<https://missinglink.ai/guides/neural-network-concepts/neural-network-bias-bias-neuron-overfitting-underfitting/>

Deep Learning – Ciclo de Vida (IX)

- Algunas cuestiones (IV):

- Algunas otras soluciones a revisar cuando aparece overfitting son:

1. Reentrenar la red con diferentes pesos. Normalmente el modelo que obtiene peor rendimiento sobre el conjunto de entrenamiento se comportará mejor sobre el conjunto de validación.
2. Entrenar múltiples redes en paralelo para quedarse con la mejor. Google ofrece la posibilidad de definir modelos variando distintos hiperparámetros para seleccionar el mejor modelo.
3. Early Stopping: Utilizando un conjunto de entrenamiento y un conjunto de validación, se controla el error en el conjunto de validación después de cada iteración de entrenamiento. Normalmente, el error de validación y los errores de la serie de entrenamiento disminuyen durante el entrenamiento, pero cuando la red empieza a sobre-ajustar (overfitting) los datos, el error en la serie de validación empieza a aumentar. La parada temprana se produce en uno de los dos casos:
 - Si el entrenamiento no tiene éxito, por ejemplo en un caso en el que la tasa de error aumenta gradualmente a lo largo de varias iteraciones.
 - Si la mejora del entrenamiento es insignificante, por ejemplo, la tasa de mejora es inferior a un umbral establecido (threshold).
4. Regularización: Solución que añade un término a la función de error, que tiene por objeto disminuir los pesos y el bias, **suavizando** los resultados y haciendo menos probable que la red se sobrepase.
 - Regularización L1: Un alto lambda lleva a underfitting. Un bajo lambda lleva a overfitting.

$$\sum_{i=1}^n (y_i - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

- Regularización L2: Un alto lambda lleva a underfitting. Un bajo lambda lleva a overfitting.

$$\sum_{i=1}^n (Y_i - \sum_{j=1}^p X_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

Deep Learning – Ciclo de Vida (X)

- Algunas cuestiones (V):

- Algunas otras soluciones a revisar cuando aparece overfitting son:

5. Suavizado o Label Smoothing: un etiquetado suave (label smoothing) es un etiquetado más permisivo, que tolera o admite una pequeña cantidad de duda, de que unos datos de entrada, a pesar de ser con total seguridad de una clase particular, quizás, en algunos escenarios, pudieran pertenecer a otra.

- airplane (avión) → 0.
- automobile (automóvil) → 1.
- bird (ave) → 2.
- cat (gato) → 3.
- deer (venado) → 4.
- dog (perro) → 5.
- frog (rana) → 6.
- horse (caballo) → 7.
- ship (barco) → 8.
- truck (camión) → 9.

→
One hot
encoding

- airplane (avión) → [1, 0, 0, 0, 0, 0, 0, 0, 0, 0].
- automobile (automóvil) → [0, 1, 0, 0, 0, 0, 0, 0, 0, 0].
- bird (ave) → [0, 0, 1, 0, 0, 0, 0, 0, 0, 0].
- cat (gato) → [0, 0, 0, 1, 0, 0, 0, 0, 0, 0].
- deer (venado) → [0, 0, 0, 0, 1, 0, 0, 0, 0, 0].
- dog (perro) → [0, 0, 0, 0, 0, 1, 0, 0, 0, 0].
- frog (rana) → [0, 0, 0, 0, 0, 0, 1, 0, 0, 0].
- horse (caballo) → [0, 0, 0, 0, 0, 0, 0, 1, 0, 0].
- ship (barco) → [0, 0, 0, 0, 0, 0, 0, 0, 1, 0].
- truck (camión) → [0, 0, 0, 0, 0, 0, 0, 0, 0, 1].

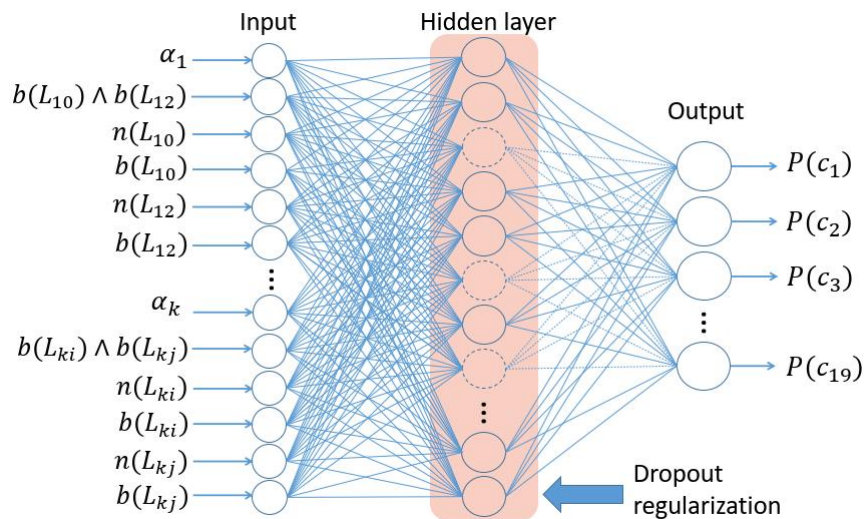
→
Label
smoothing

- airplane (avión) → [0.91, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01].
- automobile (automóvil) → [0.01, 0.91, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01].
- bird (ave) → [0.01, 0.01, 0.91, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01].
- cat (gato) → [0.01, 0.01, 0.01, 0.91, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01].
- deer (venado) → [0.01, 0.01, 0.01, 0.01, 0.91, 0.01, 0.01, 0.01, 0.01, 0.01].
- dog (perro) → [0.01, 0.01, 0.01, 0.01, 0.01, 0.91, 0.01, 0.01, 0.01, 0.01].
- frog (rana) → [0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.91, 0.01, 0.01, 0.01].
- horse (caballo) → [0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.91, 0.01, 0.01].
- ship (barco) → [0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.91, 0.01].
- truck (camión) → [0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.91].

Deep Learning – Ciclo de Vida (XI)

- Algunas cuestiones (VI):

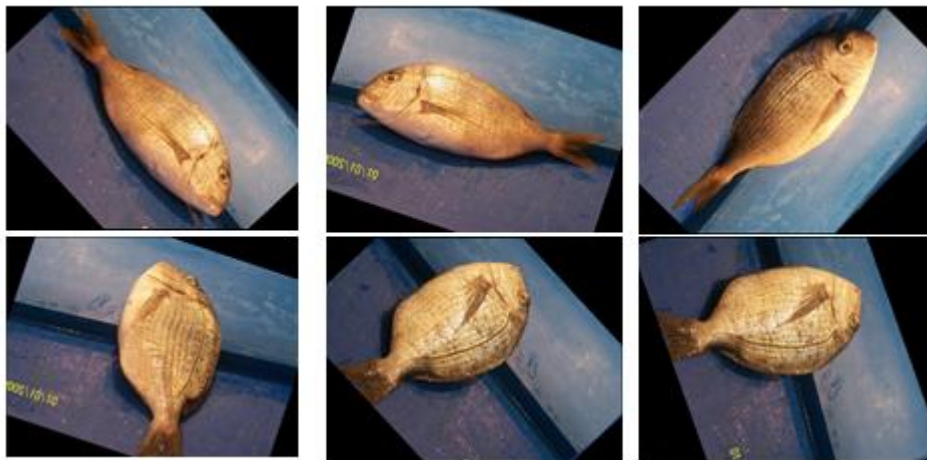
- Algunas otras soluciones a revisar cuando aparece overfitting son:
 6. Dropout: Desactivación aleatoria de ciertas neuronas de una capa.



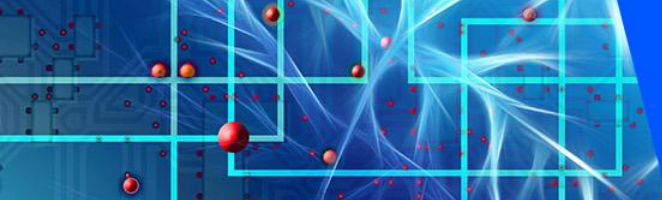
Deep Learning – Ciclo de Vida (XII)

- Algunas cuestiones (VII):

- Algunas otras soluciones a revisar cuando aparece overfitting son:
 - 7. Data augmentation: Se suele utilizar principalmente en imágenes, por ejemplo añadiendo imágenes deformadas, rotadas o con cambios de iluminación al dataset.



Large-Scale Dataset for Fish Segmentation and Classification (Ulucan, Karakaya, & Turkan, A Large-Scale Dataset for Fish Segmentation and Classification, 2020)



Deep Learning – Ciclo de Vida (XIII)

- Algunas cuestiones (VIII):

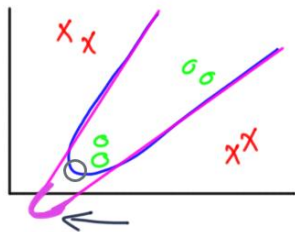
4. Underfitting: Ocurre cuando nuestro modelo no se ajusta ni al conjunto de entrenamiento ni al conjunto de validación.
 - Soluciones:
 - Añadir más capas ocultas a la red: Algunos problemas se resuelven con modelos más complejos que pueden necesitar más capas.
 - Añadir más entradas a la red: Puede ser que las entradas que hemos definido no son suficientes y necesitamos nuevos parámetros.
 - Añadir más datos de entrenamiento: Tenemos que ver si tenemos suficientes datos para entrenar la red.
 - Mejorar la calidad de los datos de entrenamiento: ¿Nuestros datos son buenos? ¿Hay valores a 0 o sin sentido?
 - Drop-out: Técnica que elimina aleatoriamente neuronas del modelo en cada iteración. Esta técnica ha demostrado ser efectiva para combatir también el overfitting al mejorar la generalización.
 - Si se ha utilizado regularización de la función de error, es posible que se haya utilizado un ratio demasiado alto y haya que reducirlo.

Deep Learning – Ciclo de Vida (XIV)

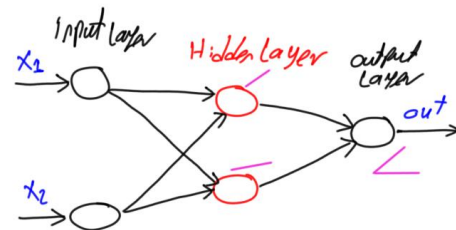
- Algunas cuestiones (IX):

5. ¿Cuántas capas y neuronas utilizar?

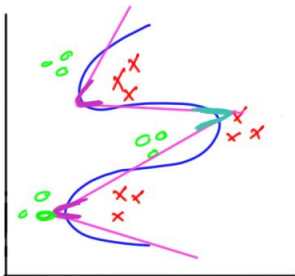
- <https://towardsdatascience.com/beginners-ask-how-many-hidden-layers-neurons-to-use-in-artificial-neural-networks-51466afa0d3e>



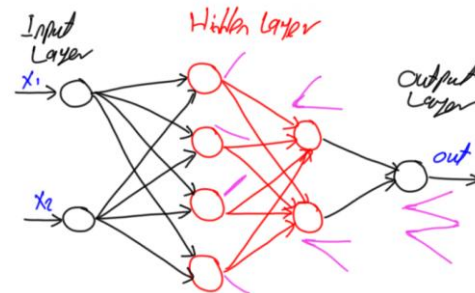
Un juego de datos que se separa por dos líneas. Cada línea corresponde con una neurona y el vértice a una capa

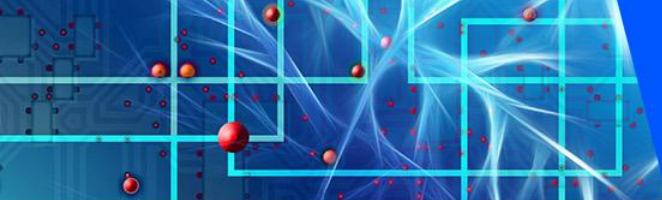


“En las redes neuronales artificiales, se requieren capas ocultas si y sólo si los datos deben separarse de forma no lineal.”



Un juego de datos que se separa por tres líneas que se cruzan. Las 4 líneas son las 4 neuronas de entrada. Los vértices en rosa corresponden a la primera capa mientras que el vértice azul, que une dos vértices rosas, es la segunda capa. El vértice azul recibe dos líneas, neuronas de la segunda capa.





Deep Learning – Ciclo de Vida (XV)

- Algunas cuestiones (X):

6. ¿Cuántas capas y neuronas utilizar?

- Rule of Thumb: Cuando tenemos una capa oculta:

$$N_h = \frac{N_s}{\alpha \cdot (N_i + N_o)}$$

N_i = Número de neuronas de entrada.

N_o = Número de neuronas de salida.

N_s = Número de muestras (samples) del conjunto de entrenamiento.

N_h = Número de neuronas de la capa oculta.

α = Factor de escala arbitrario con valor entre 2 y 10. Representa un factor de ramificación efectivo (número de pesos no nulos por neurona). Un valor 2 puede funcionar bien para evitar overfitting.

- Cuando tenemos varias capas ocultas:

$$N_s = (N_i + N_o) \cdot N_h^{N_l}$$

N_i = Número de neuronas de entrada.

N_o = Número de neuronas de salida.

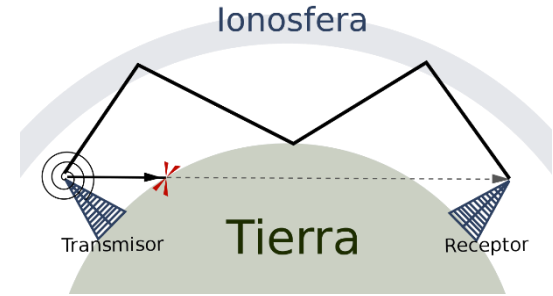
N_s = Número de muestras (samples) del conjunto de entrenamiento.

N_h = Número medio de neuronas por capa.

N_l = Número de capas ocultas.

Deep Learning – Ejercicio 1 (I)

- Clasificación binaria: Johns Hopkins University Ionosphere database
 - La ionosfera es la parte de la atmósfera terrestre ionizada permanentemente debido a la radiación solar. Se sitúa entre los 80 km y los 400 km de altitud.
 - Estos datos de radar fueron recogidos por un sistema en Goose Bay, Labrador. Este consiste en un sistema en fase de 16 antenas de alta frecuencia con una potencia total transmitida de 6.4 kilovatios. Los objetivos eran electrones libres en la ionosfera. Los "buenos" retornos de radar (good) son aquellos que muestran evidencia de algún tipo de estructura en la ionosfera. Los "malos" retornos son aquellos que no lo hacen (bad); sus señales pasan a través de la ionosfera. Las señales recibidas fueron procesadas usando una función de auto-correlación cuyas variables son el tiempo de un pulso y el número de pulso. Había 17 números de pulso para el sistema de Goose Bay. Los casos en esta base de datos están descritos por 2 atributos por número de pulsos, correspondientes a los valores devueltos por la función resultante de la señal electromagnética.
 - El problema busca detectar estructuras en la ionosfera a partir de un nuevo conjunto de datos de entrada.



Propagación de onda corta mediante rebotes en la ionosfera
<https://es.wikipedia.org/wiki/Ionosfera>

La ionosfera está relacionada con la emisión de radiaciones solares, que es el resultado del movimiento de la Tierra alrededor del sol o los cambios en la actividad del sol darán lugar a variaciones en la ionosfera que son de dos tipos generales: 1) las que son más o menos regulares y se producen en ciclos y pueden predecirse con antelación con una precisión razonable, y 2) las que son irregulares debido al comportamiento anormal del sol y, por lo tanto, no pueden predecirse con antelación.

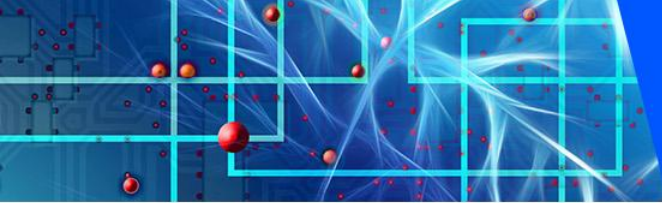
Deep Learning – Ejercicio 1 (II)

- Ejecutar en un nuevo notebook de Jupyter (a)

```
# mlp for binary classification
from pandas import read_csv
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from keras import Sequential
from keras.layers import Dense
import numpy as np
# load the dataset
path = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/ionosphere.csv'
df = read_csv(path, header=None)
# split into input and output columns
X, y = df.values[:, :-1], df.values[:, -1]
# ensure all data are floating point values
X = X.astype('float32')
# encode strings to integer
y = LabelEncoder().fit_transform(y)
# split into train and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
# determine the number of input features
n_features = X_train.shape[1]
```

Continúa





Deep Learning – Ejercicio 1 (III)

- Ejecutar en un nuevo notebook de Jupyter (b)

```
# define model
model = Sequential()
model.add(Dense(10, activation='relu', kernel_initializer='he_normal',
input_shape=(n_features,)))
model.add(Dense(8, activation='relu', kernel_initializer='he_normal'))
model.add(Dense(1, activation='sigmoid'))
# compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# fit the model
model.fit(X_train, y_train, epochs=150, batch_size=32, verbose=0)
# evaluate the model
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print('Test Accuracy: %.3f' % acc)
# make a prediction
row = [1,0,0.99539,-0.05889,0.85243,0.02306,0.83398,-0.37708,1,0.03760,0.85243,-
0.17755,0.59755,-0.44945,0.60536,-0.38223,0.84356,-0.38542,0.58212,-0.32192,0.56971,-
0.29674,0.36946,-0.47357,0.56811,-0.51171,0.41078,-0.46168,0.21266,-0.34090,0.42267,-
0.54487,0.18641,-0.45300]
yhat = model.predict(np.array([row]))
print('Predicted: %.3f' % yhat)
```

Deep Learning – Ejercicio 1 (IV)

- Resultado

Pregunta:

¿Cuántas capas intermedias
tiene nuestra red?

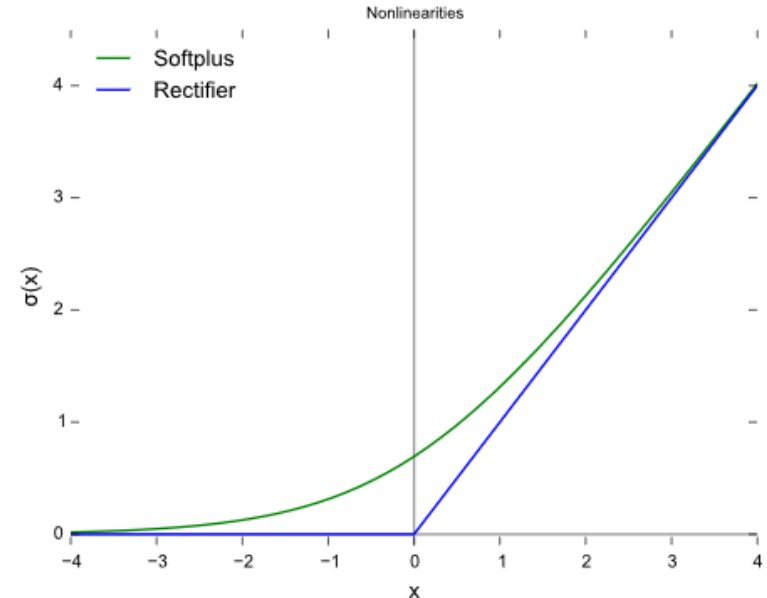
Tiene 2 capas intermedias
de 10 y 8 neuronas

```
In [9]: # mlp for binary classification
from pandas import read_csv
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from keras import Sequential
from keras.layers import Dense
import numpy as np
# load the dataset
path = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/ionosphere.csv'
df = read_csv(path, header=None)
# split into input and output columns
X, y = df.values[:, :-1], df.values[:, -1]
# ensure all data are floating point values
X = X.astype('float32')
# encode strings to integer
y = LabelEncoder().fit_transform(y)
# split into train and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
# determine the number of input features
n_features = X_train.shape[1]
# define model
model = Sequential()
model.add(Dense(10, activation='relu', kernel_initializer='he_normal', input_shape=(n_features,)))
model.add(Dense(8, activation='relu', kernel_initializer='he_normal'))
model.add(Dense(1, activation='sigmoid'))
# compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# fit the model
model.fit(X_train, y_train, epochs=150, batch_size=32, verbose=0)
# evaluate the model
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print('Test Accuracy: %.3f' % acc)
# make a prediction
row = [1,0,0,0.99539,-0.05889,0.85243,0.02306,0.83398,-0.37708,1,0.03760,0.85243,-0.17755,0.59755,-0.44945,0.60536,-0.38223,0.84356]
yhat = model.predict(np.array([row]))
print('Predicted: %.3f' % yhat)

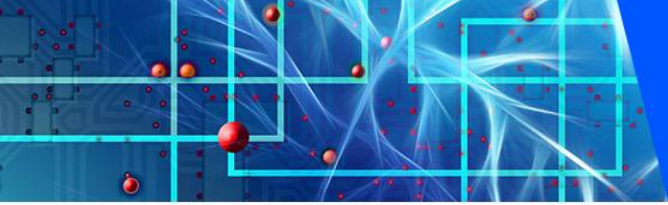
(235, 34) (116, 34) (235,) (116,)
Test Accuracy: 0.931
Predicted: 0.991
```

Deep Learning – Ejercicio 1 (V)

- Algunos comentarios (I):
 - Es buena práctica utilizar la función de activación de "relu", $f(x)=\max(0, x)$, con una inicialización de peso "he_normal". Esta combinación ayuda mucho a superar el problema de los gradientes que desaparecen cuando se entrenan modelos de redes neuronales profundas.
 - El modelo predice la probabilidad de la clase 1 y utiliza la función de activación del sigmoide. El modelo está optimizado usando la versión adam de descenso de gradiente estocástico y busca minimizar la pérdida de entropía cruzada.



Función Unidad Lineal Rectificada (ReLu) y función Softplus



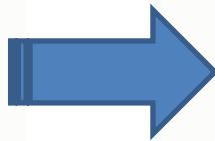
Deep Learning – Ejercicio 1 (VI)

- Modifica el programa para eliminar la capa oculta de 8 neuronas.
¿Qué resultados obtienes?
- Ahora cambia el número de epochs a 500 y vuelve a probar.
¿Qué resultado obtienes?

Muchos problemas se resuelven con una única capa oculta aunque la experimentación o análisis de los datos nos pueden dar pistas sobre cuántas capas o neuronas utilizar.

Entropía cruzada para clasificación

		NN1	NN2
	Dog	1	0.9
	Cat	0	0.1
	Dog	0	0.1
	Cat	1	0.9
	Dog	1	0.4
	Cat	0	0.6



NN1

Row	Dog [^]	Cat [^]	Dog	Cat
#1	0.9	0.1	1	0
#2	0.1	0.9	0	1
#3	0.4	0.6	1	0

Classification Error

$$1/3 = 0.33$$

Mean Squared Error

$$0.25$$

Cross-Entropy

$$0.38$$

NN2

Row	Dog [^]	Cat [^]	Dog	Cat
#1	0.6	0.4	1	0
#2	0.3	0.7	0	1
#3	0.1	0.9	1	0

$$1/3 = 0.33$$

$$0.71$$

$$1.06$$

Al principio de la retro propagación, el valor de salida que se tiene normalmente es mínimo, mucho más pequeño que el valor real deseado. El gradiente también suele ser muy bajo, lo que dificulta que la red neuronal utilice realmente los datos que tiene para ajustar los pesos. La función de entropía cruzada, a través de su logaritmo, permite a la red evaluar estos pequeños errores y trabajar para eliminarlos.

La entropía cruzada permite aumentar el error relativo en problemas de clasificación

Deep Learning – Ejercicio 2 (I)

- Clasificación multi-clase:
Planta Iris

- Se busca clasificar el tipo de planta Iris a partir de un conjunto de datos que contiene 3 clases de 50 instancias cada una, donde cada clase se refiere a un tipo de planta de iris. Una clase es linealmente separable de las otras dos; estas últimas no son linealmente separables entre sí.
- A partir de un conjunto de los datos de una nueva planta se busca predecir la clase de planta de iris.
- Los datos del dataset son:
 1. Longitud del sépalo en cm
 2. Anchura del sépalo en cm
 3. Longitud del pétalo en cm
 4. Anchura del pétalo en cm
 5. Clase: Setosa/Versicolor/Virginica



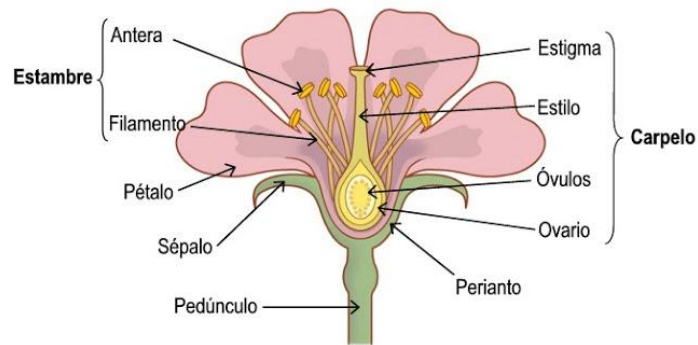
Iris Setosa



Iris Versicolor



Iris Virginica



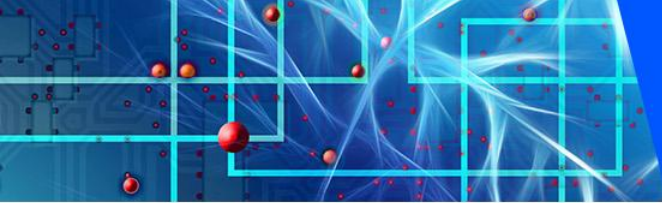
Deep Learning – Ejercicio 2 (II)

- Ejecutar en un nuevo notebook de Jupyter (a)

```
# mlp for multiclass classification
from numpy import argmax
from pandas import read_csv
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from keras import Sequential
from keras.layers import Dense
import numpy as np
# load the dataset
path = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv'
df = read_csv(path, header=None)
# split into input and output columns
X, y = df.values[:, :-1], df.values[:, -1]
# ensure all data are floating point values
X = X.astype('float32')
# encode strings to integer
y = LabelEncoder().fit_transform(y)
# split into train and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
# determine the number of input features
n_features = X_train.shape[1]
```

Continúa





Deep Learning – Ejercicio 2 (III)

- Ejecutar en un nuevo notebook de Jupyter (b)

```
# define model
model = Sequential()
model.add(Dense(10, activation='relu', kernel_initializer='he_normal',
input_shape=(n_features,)))
model.add(Dense(8, activation='relu', kernel_initializer='he_normal'))
model.add(Dense(3, activation='softmax'))
# compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
# fit the model
model.fit(X_train, y_train, epochs=150, batch_size=32, verbose=0)
# evaluate the model
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print('Test Accuracy: %.3f' % acc)
# make a prediction
row = [5.1, 3.5, 1.4, 0.2]
yhat = model.predict(np.array([row]))
print('Predicted: %s (class=%d)' % (yhat, argmax(yhat)))
```

Deep Learning – Ejercicio 2 (IV)

- Resultado

Pregunta:

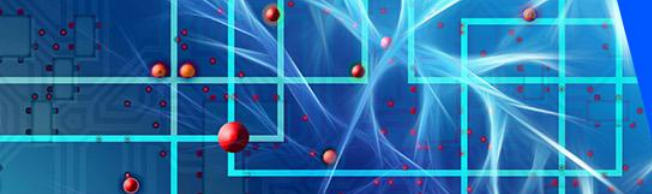
¿Cuántas neuronas tiene la
capa de salida de nuestra red?

Tiene 3 neuronas, una por cada
clase de salida

```
In [3]: # mlp for multiclass classification
from numpy import argmax
from pandas import read_csv
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from keras import Sequential
from keras.layers import Dense
import numpy as np

# Load the dataset
path = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv'
df = read_csv(path, header=None)
# split into input and output columns
X, y = df.values[:, :-1], df.values[:, -1]
# ensure all data are floating point values
X = X.astype('float32')
# encode strings to integer
y = LabelEncoder().fit_transform(y)
# split into train and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
# determine the number of input features
n_features = X_train.shape[1]
# define model
model = Sequential()
model.add(Dense(10, activation='relu', kernel_initializer='he_normal', input_shape=(n_features,)))
model.add(Dense(8, activation='relu', kernel_initializer='he_normal'))
model.add(Dense(3, activation='softmax'))
# compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
# fit the model
model.fit(X_train, y_train, epochs=150, batch_size=32, verbose=0)
# evaluate the model
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print('Test Accuracy: %.3f' % acc)
# make a prediction
row = [5.1, 3.5, 1.4, 0.2]
yhat = model.predict(np.array([row]))
print('Predicted: %s (class=%d)' % (yhat, argmax(yhat)))

(100, 4) (50, 4) (100,) (50,)
Test Accuracy: 0.980
Predicted: [[ 0.924806   0.02482159  0.05037253]] (class=0)
```

Deep Learning – Ejercicio 2 (V)

- Como en el ejercicio anterior, modifica el programa para eliminar la capa oculta de 8 neuronas. ¿Qué resultados obtienes?
- Ahora cambia el número de epochs a 500 y vuelve a probar. ¿Qué resultado obtienes?
- Ahora prueba a modificar la función relu por una función softplus. ¿Qué resultado obtienes?

Muchos problemas se resuelven con una única capa oculta, aunque la experimentación o análisis de los datos nos pueden dar pistas sobre cuántas capas o neuronas utilizar.

Deep Learning – Ejercicio 3 (VII)

• Resultado

Pregunta:

¿Cuántas capas ocultas

tiene nuestra red?

Tiene una capa de convolución, una capa de pooling, una capa flatten, una Full-Connected (Dense), una Dropout y otra Full-Connected (Dense)

```
In [8]: # example of a cnn for image classification
from numpy import unique
from numpy import argmax
from keras.datasets.mnist import load_data
from keras import Sequential
from keras.layers import Dense
from keras.layers import Conv2D
from keras.layers import MaxPool2D
from keras.layers import Flatten
from keras.layers import Dropout
import numpy as np
# Load dataset
(x_train, y_train), (x_test, y_test) = load_data()

# reshape data to have a single channel
x_train = x_train.reshape((x_train.shape[0], x_train.shape[1], x_train.shape[2], 1))
x_test = x_test.reshape((x_test.shape[0], x_test.shape[1], x_test.shape[2], 1))

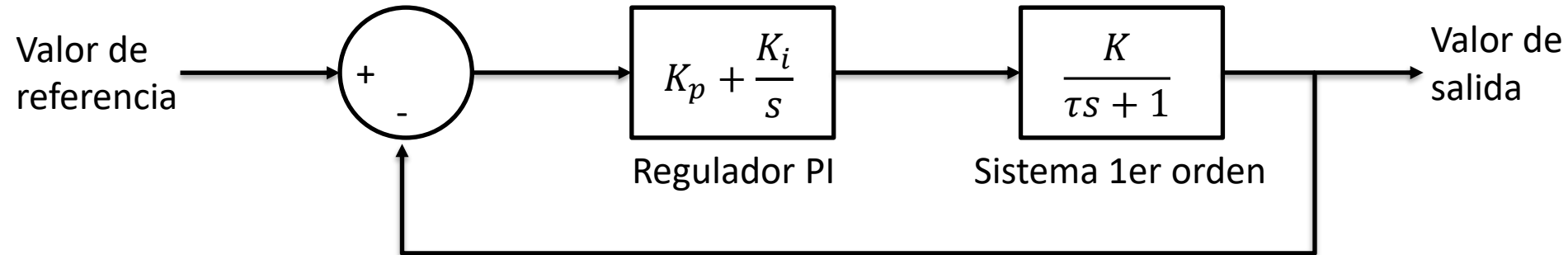
# Reducimos el dataset para que vaya más rápido (1/10)
x_train = x_train[:6000, :, :, :]
y_train = y_train[:6000]
x_test = x_test[:1000, :, :, :]
y_test = y_test[:1000]

# determine the shape of the input images
in_shape = x_train.shape[1:]
# determine the number of classes
n_classes = len(unique(y_train))
print(in_shape, n_classes)
# normalize pixel values
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
# define model
model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', input_shape=in_shape))
model.add(MaxPool2D((2, 2)))
model.add(Flatten())
model.add(Dense(100, activation='relu', kernel_initializer='he_uniform'))
model.add(Dropout(0.5))
model.add(Dense(n_classes, activation='softmax'))
# define loss and optimizer
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
# fit the model
model.fit(x_train, y_train, epochs=10, batch_size=128, verbose=0)
# evaluate the model
loss, acc = model.evaluate(x_test, y_test, verbose=0)
print('Accuracy: %.3f' % acc)
# make a prediction
image = x_train[0]
yhat = model.predict(np.array([image]))
print('Predicted: class=%d' % argmax(yhat))

(28, 28, 1) 10
Accuracy: 0.965
Predicted: class=5
```

Regulador mediante MLP (I)

- Supongamos que queremos simular mediante una red neuronal el comportamiento de un sistema dinámico de primer orden al que hemos añadido un regulador de tipo proporcional-integrador. El sistema opera por lo tanto en lazo cerrado.



Regulador mediante MLP (II)

- Primer paso: Función de simulación del sistema dinámico.

```
import numpy as np
import control as ctrl
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Función para simular el sistema de primer orden con regulador PI
# El sistema parte de 0. reference es el valor de salida a alcanzar
def simulate_system(K, tau, Kp, Ki, reference, t):
    G = ctrl.TransferFunction([K], [tau, 1]) # FT  $K/(tau*s + 1)$ 
    C = ctrl.TransferFunction([Kp, Ki], [1, 0]) # Regulador:  $Kp + Ki/s$ 
    FLC = ctrl.feedback(C * G, 1)

    # Vector con el valor de salida esperado a lo largo del tiempo
    U = np.ones_like(t) * reference

    _, y = ctrl.forced_response(FLC, t, U)
    return y
```

Regulador mediante MLP (II)

- Segundo paso:
Generamos 30,000 simulaciones con distintos parámetros de K_p , K_i , K , τ , valor de referencia.

```
# Generar datos de entrenamiento
def generate_training_data(num_samples):
    data = []
    labels = []

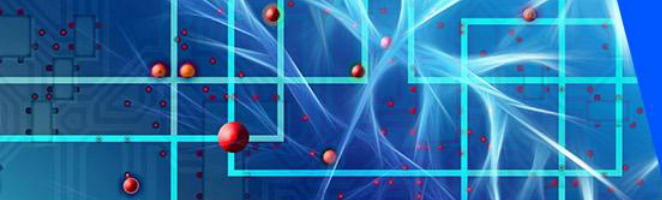
    # Para cada una de las repeticiones creamos una simulación con
    # los parámetros dados
    for _ in range(num_samples):
        K = np.random.uniform(0.5, 1.5)
        tau = np.random.uniform(0.5, 1.5)
        Kp = np.random.uniform(0.5, 1.5)
        Ki = np.random.uniform(0.5, 1.5)
        referencia = np.random.uniform(0, 2)

        # Vector de tiempo de la simulación
        t = np.linspace(0, 10, 100)
        y = simulate_system(K, tau, Kp, Ki, referencia, t)

        # Añadimos los datos al juego de entrenamiento
        for i in range(len(t) - 1):
            data.append([K, tau, Kp, Ki, y[i], referencia])
            labels.append(y[i + 1])

    return np.array(data), np.array(labels)

num_samples = 30000
data, labels = generate_training_data(num_samples)
```



Regulador mediante MLP (III)

- Dividimos los datos en train/test.
- Definimos el modelo.
- Entrenamos y evaluamos.

```
# Dividir los datos en conjuntos de entrenamiento y prueba
train_size = int(0.8 * num_samples)
train_data, test_data = data[:train_size], data[train_size:]
train_labels, test_labels = labels[:train_size], labels[train_size:]

# Crear el modelo de la red neuronal
model = Sequential()
model.add(Dense(256, input_dim=6, activation='relu'))
model.add(Dense(256, activation='relu'))
model.add(Dense(256, activation='relu'))
model.add(Dense(1, activation='linear'))

# Compilar el modelo
opt = tf.keras.optimizers.Adam(learning_rate=0.00005)
model.compile(optimizer=opt, loss='mse')

# Entrenar el modelo
historia = model.fit(train_data, train_labels, epochs=300, batch_size=128,
                    validation_split=0.2)

# Evaluar el modelo
loss = model.evaluate(test_data, test_labels)
print(f'Pérdida en el conjunto de prueba: {loss}')
```

Regulador mediante MLP (IV)

- Representamos la pérdida del entrenamiento /validación.
- Definimos unos valores para una simulación de ejemplo y obtenemos sus valores con la librería de control (será nuestro ground-truth).

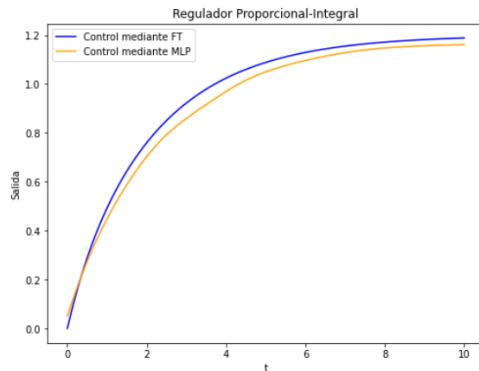
```
# Representamos gráficamente el entrenamiento mostrando el error
plt.plot(historia.history['loss'], 'r', label='Train Loss')
plt.plot(historia.history['val_loss'], 'c', label='Validation Loss')
plt.ylabel('Loss')
plt.xlabel('Época')
plt.title("Evolución del loss")
plt.legend(loc='center right')
plt.show()

# Simulación del sistema mediante Lazo Cerrado y mediante Red Neuronal
# datos de ejemplo
K = 0.7
tau = 0.6
Kp = 0.5
Ki = 0.7
referencia = 1.2

# Creamos un vector de tiempo
t = np.linspace(0, 10, 100)
# Obtenemos valores del control en lazo cerrado (Nuestro ground truth)
y_real = simulate_system(K, tau, Kp, Ki, referencia, t)
```

Regulador mediante MLP (V)

- Realizamos la simulación con la red neuronal y visualizamos su comparación con la simulación de la librería de control



```
# Hacemos inferencia con la red para los valores del controlador
y_predicted = []

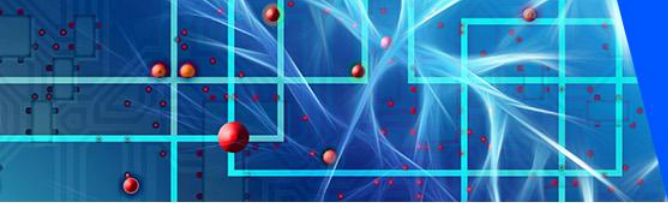
y_actual = 0 # partimos de 0
for ti in t:
    predicted_output = model.predict(np.array([[K, tau, Kp, Ki, y_actual,
referencia]]))
    y_actual = predicted_output[0][0]
    y_predicted.append(y_actual)

# Representamos ambos resultados
# Crear una figura y un objeto Axes
fig, ax = plt.subplots(figsize=(8, 6))

# Primer gráfico
ax.plot(t, y_real, label='Control mediante FT', color='blue')
# Segundo gráfico
ax.plot(t, y_predicted, label='Control mediante MLP', color='orange')

# Configurar el título, las etiquetas y la leyenda
ax.set_title('Regulador Proporcional-Integral')
ax.set_xlabel('t')
ax.set_ylabel('Salida')
ax.legend()

# Mostrar la gráfica
plt.show()
```

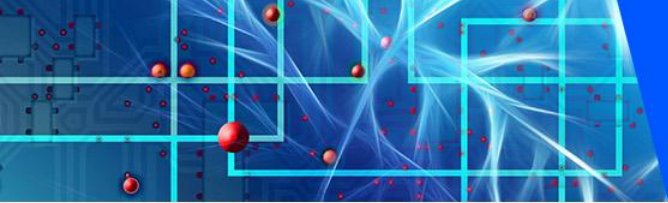



Algunas preguntas (I)

1. En el aprendizaje automático, algunos métodos de clasificación son:
 - a) SVM, redes neuronales, PCA y Random Forest.
 - b) Árboles de decisión, SVM, PCA y Extra Trees.
 - c) PCA, SVM y boosting.
 - d) Ninguna de las otras opciones.

2. Si el tamaño de nuestro dataset es de 16.000 registros y hemos elegido un tamaño de batch=16, ¿cuántas épocas definiremos en el entrenamiento de una red neuronal?
 - a) 1.000
 - b) 16.000
 - c) 16
 - d) Cualquiera de las opciones anteriores podría ser correcta.

3. ¿Cuál es una de las ventajas de SVM respecto a una red neuronal?
 - a) Tiene menos hiperparámetros.
 - b) Es mucho más rápido el proceso de entrenamiento.
 - c) Permite resolver problemas de clasificación.
 - d) Todas las otras opciones son correctas.



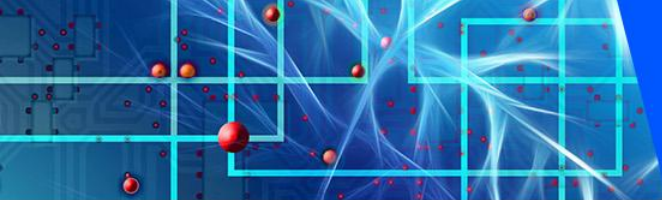
Algunas preguntas (II)

4. ¿Cómo se denomina el proceso en el que terminamos el entrenamiento de un modelo cuando algún indicador comienza a empeorar, por ejemplo, validation accuracy?

- a) Early-Stopping.
- b) Batch-Generation.
- c) Block-Training.
- d) Ninguna de las otras opciones es correcta.

5. Dentro de los métodos de regularización, podemos encontrar:

- a) Dropout.
- b) Suavizado.
- c) Regularización L1.
- d) Todas las otras opciones son correctas.



Algunas preguntas (III)

6. Cuando en un modelo tenemos un test accuracy del 90% y un balanced test accuracy del 50%, ¿qué está ocurriendo?
- a) El dataset está balanceado.
 - b) El dataset no está balanceado.
 - c) El dataset tiene una única clase.
 - d) Ninguna de las otras opciones es correcta.
7. En el aprendizaje automático, algunos métodos de regresión son:
- a) SVM, redes neuronales, PCA y random forest.
 - b) Árboles de decisión, SVM, PCA y Extra Trees.
 - c) PCA, SVM y boosting.
 - d) SVM, redes neuronales, Random Forest y Extra Trees.

Ejercicios adicionales (I)

El dataset del vino (<https://github.com/zygmuntz/wine-quality/blob/master/winequality-red.csv>) analiza la calidad del vino en función de distintos parámetros.

Las variables de entrada son:

- 1 - fixed acidity
- 2 - volatile acidity
- 3 - citric acid
- 4 - residual sugar
- 5 - chlorides
- 6 - free sulfur dioxide
- 7 - total sulfur dioxide
- 8 - density
- 9 - pH
- 10 - sulphates
- 11 - alcohol

La variable de salida es:

- 12 - quality (score between 0 and 10)



<https://pixabay.com/photos/wine-bottle-of-wine-red-wine-cup-4813260/>

Se pide:

1. Cargar el dataset en memoria.
2. Dividir los datos en: 70% train, 20% validación, 10% test. Hacer un reparto aleatorio.
4. Entrenar una red neuronal cuya entrada sean las 11 variables de entrada y cuya salida sean 10 posibles calidades (clasificación). (En Tensorflow)
5. Representar la gráfica del entrenamiento.
6. Evaluar el modelo con test.
7. Intenta añadir capas, neuronas, modificar la función de activación, dropout, o cambiar la función de error para intentar mejorar los resultados.
8. Transforma tu programa Tensorflow en Pytorch. Intenta ejecutarlo.

Ejercicios adicionales (II)

Cálculo del determinante de una matriz

Genera un conjunto de datos donde la entrada esté compuesta por los valores de una matriz de 3x3. Cada uno de los valores será de tipo entero. La salida del modelo será el determinante de la matriz.

Intenta aproximar el cálculo del determinante de una matriz creando una red cuya entrada sean los 9 elementos de la matriz y cuya salida sea el valor del determinante.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = a_{11} * a_{22} * a_{33} + a_{12} * a_{23} * a_{31} + a_{13} * a_{21} * a_{32} \\ - a_{31} * a_{22} * a_{13} - a_{32} * a_{23} * a_{11} - a_{33} * a_{12} * a_{21}$$